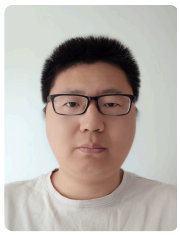
# 张恒

**求职意向：C/C++ 高级软件工程师**


工作年限：16 年
期望城市：青岛
电　　话：151-9248-5800
邮　　箱：354379271@qq.com

## 教育背景

**大连理工大学（985）** | 软件工程（嵌入式方向） | 本科 | 2006.09 – 2010.06 | 英语六级

## 个人优势

- **16 年 C/C++ 开发经验**，其中 11 年 Linux 高并发服务端开发，5 年 ARM/VxWorks/FreeRTOS 嵌入式实时系统开发
- 擅长高并发服务端与嵌入式实时系统的**架构设计**，熟练运用多线程、多进程、共享内存与 epoll 事件驱动模型
- 长期从事**性能调优**，能定位并优化 CPU、内存与网络 I/O 瓶颈，有 NUMA、睿频、中断亲和与 AVX2 SIMD 层面的调优经验
- 擅长复杂故障根因分析，用抓包、perf 与控制变量实验定位过 SCTP 长连接时延、KVM 虚拟网卡多队列乱序、kubelet CPU 分配不均等问题；2 年北美、欧洲运营商客户支持经验
- 主导 C++ 服务的 **Docker/Kubernetes** 容器化改造，设计优雅启停、新旧版本兼容与滚动升级方案，实现业务不中断升级
- 英语可作为工作语言，多次牵头跨团队新功能开发与技术攻关
- 业余从零实现 C++20 大模型 CPU 推理引擎（AVX2 + FMA 矩阵乘达本机实测峰值 66%），并开发过 AI Agent 与 RAG 应用

## 工作经历

### 诺基亚通信投资（中国）有限公司青岛分公司 · 高级软件工程师　2015.06 – 至今

| | |
|---|---|
| **高性能后端** | 负责 4G/5G 核心网用户数据库（HSS/UDM）Linux C++ 后端服务开发，基于 epoll、多线程与共享内存实现 Diameter、HTTP/2 请求的高并发、低时延处理 |
| **模块负责** | 网络通信、数据库缓存、通用应用中间件 3 个核心模块负责人，负责架构设计与代码评审 |
| **云原生改造** | 主导用户数据库（HSS）后端服务器的 Docker/Kubernetes 容器化改造，负责全产品不中断升级方案设计 |
| **客户支持** | 为北美、欧洲运营商定位并解决核心网现网的网络、CPU、内存泄漏等重大问题（2018.10 – 2020.09） |
| **团队带领** | 多次牵头新功能开发，带领 6 ~ 7 人 Scrum 小组从设计到交付；负责新员工培训与技术分享 |

### 日冲信息（大连）有限公司 · 嵌入式软件工程师　2010.06 – 2015.05

| | |
|---|---|
| **网络协议栈** | 依据 RFC 在 ARM/VxWorks/FreeRTOS 上用 C/C++ 开发嵌入式 Web 服务器及 HTTP、UPnP、SNMP、CIFS 文件共享、SMTP/POP3 邮件等网络协议，运行于内存与 CPU 受限的打印机设备 |
| **打印扫描** | 主导 AirPrint、IPP over USB、谷歌云打印与 HP eSCL 扫描协议开发，连续通过苹果 AirPrint 1.1 ~ 1.5 认证 |
| **性能优化** | 用 WindView 分析 VxWorks 多任务瓶颈，结合 Wireshark 调优 TCP/UDP 窗口与重传超时，提升传输速率 |
| **IPv6 认证** | 主持搭建 IPv6 Ready Logo 测试环境，网络协议栈通过 Phase-1/Phase-2 认证 |
| **界面开发** | 开发打印机触摸屏 GUI，用 HTML、JavaScript 等开发打印机内嵌管理网页，并用 Selenium 做自动化测试 |

## 专业技能

| | |
|---|---|
| **编程语言** | 熟练掌握 C/C++（C++11/14/17/20），熟练使用 Python、Golang、Shell |
| **系统编程** | 掌握 Linux 多线程、多进程、进程间通信与同步机制，以及 epoll 事件驱动与线程池模型 |
| **性能调优** | 擅长 CPU、内存与网络 I/O 调优，有 NUMA、中断亲和与 SIMD 优化经验；熟练使用 gdb、perf、Valgrind |
| **后端开发** | 掌握高并发服务端架构设计，包括缓存、负载均衡与高可用；生产中使用 PostgreSQL、Redis、gRPC/Protobuf |
| **网络协议** | 深入理解 TCP/IP（拥塞控制、重传机制）、HTTP/2、SCTP 等协议，Socket 编程熟练 |
| **云原生** | 熟练运用 Docker、Kubernetes、Helm，主导过容器化改造与业务不中断升级；能用 Golang 开发 CRD 控制器 |
| **嵌入式** | 5 年 ARM/VxWorks/FreeRTOS 经验，掌握多任务调度、消息队列、信号量与优先级继承，做过 USB 设备端开发 |
| **AI 技术** | 开发过 AI Agent 与 RAG 应用，理解大模型推理原理与性能优化方法；熟练使用 Cursor、Trae 等 AI 编程工具 |
| **工程工具** | 熟练使用 CMake、Git、GoogleTest、Jenkins 等开发与持续集成工具 |

## 项目经历

### 基于共享内存的高并发数据库缓存系统 · 模块负责人 2025.09 – 2025.12

**背景** 用户数据库前端上百个业务进程各自从后端数据库加载配置数据，启动与刷新时数据库压力大、CPU 过载，且进程间数据不一致

**我的工作**
- 架构设计：新增独立缓存进程统一加载配置数据并写入共享内存，业务进程从共享内存同步本地副本，数据库查询由每进程一份变为全节点一份
- 通知接入：用 C++ 开发高性能 HTTP Server（epoll + 线程池）接收数据库变更通知，双 Pod Active-Active 部署并兼做负载均衡
- 一致性保障：收到变更通知后只重新缓存变更条目或整表，再通知业务进程刷新本地副本；周期刷新与每日定点刷新作为兜底
- 质量把关：主导设计与代码评审，整理共享内存缓存的排障方法，用于生产问题定位与新员工培训

**成果** HTTP Server 达 2500 TPS；数据库查询量降低 40 倍，消除 CPU 过载；已在 T-Mobile、Verizon 商用

### C++ 后端服务容器化改造与不中断升级 · 主导开发 2020.06 – 至今

**背景** 电信核心网用户数据库由 7 个 Linux C++ 后端服务组成，原以虚拟机交付、升级需停机维护；需迁移至 Kubernetes/OpenShift，并实现升级过程业务不中断

**我的工作**
- 服务改造：牵头认证处理、数据变更通知等 C++ 核心服务的容器化改造，进程模型改为单容器多进程，配置改为 ConfigMap 注入与热加载
- 优雅启停：用 C++ 实现 SIGTERM 信号处理、长连接摘除与在途请求排空，以及就绪/存活探针，保证滚动升级时请求不丢失
- 版本兼容：设计新旧版本消息与数据格式兼容方案，支持升级期间新旧版本 Pod 混跑
- 发布编排：用 Golang 独立开发 Kubernetes CRD 自定义控制器，管理负载均衡 Pod 的滚动更新；参与全产品 8 个 Helm Chart 的发布编排与联调

**成果** 升级由 5 小时停机窗口变为不停机滚动升级，升级期间请求成功率 99.99%，单 Pod 2500 QPS；已在 T-Mobile、Verizon、Vodafone 等欧美运营商商用

### Kubernetes 容器 CPU 性能不均根因分析与调优 · 主导分析与调优 2025.09 – 2025.10

**背景** 新版本容器平台压测中，同一节点上 8 个业务 Pod 承载相同流量，CPU 占用却相差一倍以上（1430m vs 600m），影响容量评估与交付

**我的工作**
- 逐项排除：通过负载均衡计数与监控数据复核，排除流量分发不均与监控误差
- 访存排查：用 numactl、Intel PCM 量化跨 NUMA 访存，并用 Kubernetes 拓扑管理器做对照验证，证明跨 NUMA 不是主因
- 定位根因：设计十余组控制变量测试（Turbo 睿频、中断亲和、电源策略、Pod 分布等），定位主因为 kubelet CPU 分配使 Pod 在两个 NUMA 节点上分布不均，高负载节点睿频更低（2.6 vs 2.9 GHz），网卡中断集中进一步加剧
- 方案：推荐负载均衡按 Pod CPU 使用率分发流量，向 Kubernetes 社区提交 Issue 反馈 kubelet 分配问题；评估预留 CPU、关闭睿频等方案的代价

**成果** 平衡 NUMA 负载后，Pod 间 CPU 差异由约 600m 降至 150m；关闭 Turbo 可降至 100m 以内，但性能下降 10%~15%

### 大规模现网长连接时延故障根因定位（T-Mobile） · 攻关负责人 2019.06 – 2019.09

**背景** 运营商光纤中断后流量绕路，后端服务间 SCTP 长连接时延升至 140 ms 以上，约 229 万用户受影响，语音呼叫量下降 25%

**我的工作**
- 带队攻关：作为技术负责人牵头跨团队攻关，与美国、印度同事协作，协调客户现场支持与本地测试团队
- 止损：与客户协同分析日志与抓包，隔离高时延链路并配合流量迁移，先行恢复业务
- 定位：用 tcpdump、perf 对比正常与异常连接，发现发送队列堆积并出现 EAGAIN 错误，确认单线程阻塞发送使一条慢连接拖垮同线程其他连接
- 根因：组织实验室按同等时延复现，控制变量测试网卡队列、发送缓冲区与物理机/虚拟机，锁定 KVM 虚拟网卡多队列导致报文乱序与快速重传，高时延下拥塞窗口难以恢复
- 修复：用 C++ 实现基于滑动窗口的连接级自动隔离（5 秒窗口内累计 10 次写失败即摘除该连接，流量分摊到其余 23 条连接，10 秒后自动重连）；调大发送缓冲区、缩短 epoll 超时，避免单连接阻塞整个线程

**成果** 12 小时内恢复业务，72 小时内补丁被现网采纳；网卡改为单队列后单连接吞吐提升约 10 倍；推动建立广域网时延测试环境与链路级监控告警

### 苹果 AirPrint 打印协议开发与认证（ARM/VxWorks） · 主导开发 2013.11 – 2015.05

**背景** 产品线需支持 iPhone/iPad/macOS 免驱动无线打印并通过苹果官方认证，设备运行在内存不超过 256 MB、CPU 受限的 ARM/VxWorks 平台上

**我的工作**
- 服务发现：基于 Bonjour（mDNS/DNS-SD）实现打印服务的发现与通告

- 作业处理：基于 IPP 实现作业提交、状态查询与错误处理，文档数据边收边处理，支持 PDF、JPEG、URF 等格式的打印流水线
- 问题攻关：认证测试中小文档能正常打印，大 PDF 却失败或卡住；用 Wireshark 抓包发现 iOS 以 HTTP 分块传输（chunked）发送文档，而嵌入式 HTTP 服务器只支持 Content-Length，于是为其增加分块解码，边收边交给打印流水线，最终通过认证
- 认证测试：搭建认证环境，执行功能、互操作与回归测试，跟进 1.1 ~ 1.5 规范差异并修复兼容问题

**成果** 6 款打印机连续通过 AirPrint 1.1 ~ 1.5 认证，无线打印能力随产品迭代上市商用

**基于 STM32/FreeRTOS 的打印机 IPP over USB 开发** · 主导开发与测试 2013.11 – 2013.12

**背景** 传统 USB 打印依赖主机驱动、状态信息少；IPP over USB 在 USB 打印机类接口上承载 HTTP/IPP，在 STM32 针式打印机等机型上实现免驱动打印

**我的工作**

- USB 接口：基于 STM32 片上全速 USB（12 Mbps）配置 2 个打印机类接口（一路传作业、一路查状态），各配一对批量端点，并保留传统打印接口兼容老主机
- 中断处理：USB 中断只做最少的工作，把收到的数据包写入预分配的静态缓冲池，凑满一块或收到短包后通过消息队列交给对应接口的任务；USB 中断优先级设在 FreeRTOS 可管理范围内，保证在中断里调用系统接口安全
- 流控反压：缓冲池耗尽时不再重新使能 OUT 端点，由硬件自动回 NAK 让主机暂停发送，任务归还缓冲区后再恢复接收，在仅 128 KB RAM 下既不丢包也不溢出
- 报文解析：每个接口一个独立任务，用状态机解析 HTTP 报文边界（Content-Length 与 chunked），报文体边收边交给打印流水线；共享作业状态用带优先级继承的互斥信号量保护

**成果** 打印机在 USB 直连下实现免驱动打印与作业状态查询；随 6 款机型上市

**C++20 大模型 CPU 推理引擎** · 个人项目 2026.04 – 2026.06

**背景** 不依赖任何推理框架，在无 GPU 的笔记本 CPU 上从零实现 GPT-2 推理，逐项量化分析 cache、SIMD、内存带宽与线程同步对性能的影响

**我的工作**

- 矩阵乘优化：通过循环重排、三层分块、AVX2 + FMA 微内核与多线程并行，矩阵乘从 0.6 提升到 157.3 GFLOPS（274 倍，达本机实测峰值的 66%）
- 解码优化：KV Cache 采用零拷贝视图，避免每步重复拷贝；解码阶段改为按列并行的矩阵向量乘（GEMV），用常驻线程池加动态抢块适配大小核，解决了线程越多越慢的问题
- 正确性：手写前向与计算图执行两套路径互为校验，并以 numpy 为独立参考逐层比对，贪心输出与 Python 逐字节一致；126 个测试用例，ASan/UBSan/TSan 全部通过

**成果** GPT-2 解码速度由 7.5 提升到约 111 token/s（约 15 倍）

**AI 智能运维终端** · 独立开发 2026.07 – 2026.07

**背景** 运维排障需在多台主机间反复敲命令，历史工单中的处理经验也难以复用

**我的工作**

- 终端与 Agent：基于 Electron + TypeScript 开发内置 AI 侧边栏的 SSH 终端，实现基于 Function Calling 的 ReAct Agent 与 30 余个运维工具，危险命令需人工审批后执行
- 工单检索：集成工单问答系统的 RAG 能力，增量同步 Jira 工单，用内部 Ollama 部署的千问 Embedding 向量化后存入 Chroma，向量与 BM25 双路召回、RRF 融合排序，排障时参考历史处理方案

**成果** 获诺基亚内部 AI 创新大赛二等奖

## 专利、荣誉与证书

**专利** 第一发明人，PCT 国际专利 WO2025175539A1（《基于设备信息的 AKMA 认证》）

**荣誉** 2021.10 因绩效突出获授诺基亚 1000 股股权激励

**软考** 软件设计师（中级）

**红帽认证** OpenShift 应用开发者（证书编号 240-240-408）